# VGG-T$^3$：可扩展离线前馈三维重建

历史论文按新提示词重制版 · CVPR 2026 · 2026-09-30

对应离线阅读包：解压ZIP后双击 index.html。本PDF包含完整13项学习简报，并重点展开创新点、核心算法、数学公式和固定commit源码；完整原文及逐节译文在ZIP中。

## 1. 基本信息

| | |
|---|---|
| 论文 | VGG-T$^3$: Offline Feed-Forward 3D Reconstruction at Scale |
| 作者/机构 | Sven Elflein、Ruilong Li、Sérgio Agostinho、Zan Gojcic、Laura Leal-Taixé、Qunjie Zhou、Aljosa Osep；NVIDIA、University of Toronto、Vector Institute |
| 会议/版本 | CVPR 2026；arXiv:2602.23361v1（2026-02-26），20页 |
| 资源 | 论文 · 项目 · 代码固定版本 · 权重 |

标签：3D重建、VGGT、Test-Time Training、线性注意力、多视图几何、相机位姿、深度估计、视觉定位、高效推理。

## 2. 个性化推荐理由

你的VGGT + Depth Pro + YOLO流程受多帧全局推理的显存与时延约束。VGG-T$^3$不改变主要输出协议，而是把VGGT全局softmax层改成快速权重MLP；官方infer()仍返回pose、intrinsics、pts3d、conf、depth。它补足“如何把离线全局几何扩展到数百/上千帧”的核心知识，也适合做现有pipeline的低改动替换实验。

一句话问题：怎样让无序多视图重建保留全局场景聚合，却不为每对token显式计算O(n$^2$)注意力?

## 3. 方法流程

- 输入N张RGB，缩放并patch化；默认每图为37×37 patch及camera/register token。
- 保留VGGT逐帧attention；每个全局层投影Q/K/V。
- 把V的patch恢复为二维网格，以3×3 ShortConv2D得到V′。
- Update：用K→V′自监督目标更新每头SwiGLU快速权重；长序列增加内层步数。
- Apply：用更新后的MLP处理Q，得到全局聚合token。
- 最终头输出位姿、内参、深度、置信度与点图；查询新图时冻结快速权重，仅Apply。

## 4. 真正创新点：已有方法 vs 本文

| 模块 | 旧方法 | 瓶颈 | 本文改变 | 证据 |
|---|---|---|---|---|
| 场景记忆 | VGGT保留全部KV token | 读取O(n$^2$)，显存随token增长 | 把K→V映射写入固定大小MLP | 1k图约58秒，VGGT>11分钟 |
| TTT目标 | 同token线性K→V | 可能出现平凡线性映射 | 对V做3×3 ShortConv2D | CD 0.074→0.066，mAA 72.16→74.14 |
| 长序列 | 固定一次测试更新 | 1k图误差可增约5倍 | 按长度增加内层步数 | 20图一步足够，1k图受益于多步 |
| 部署 | softmax需全序列QKV | 单卡OOM、多卡通信大 | 局部梯度求和，支持分块/卸载/多卡 | 2k图4卡48.5秒；VGGT 1590.2秒 |

沿用而非创新：DINOv2 tokenizer、VGGT预测头、SwiGLU、Muon、RoPE及TTT思想均来自既有工作。本文的新意是无序多视图的固定参数场景表示、ShortConv2D视觉目标、长序列测试时扩展与可查询地图。

## 5. 核心算法一：快速权重MLP的Update/Apply

衔接：替代VGGT每个全局attention，输入输出形状不变，后续block和预测头继续工作。

形状：x:[B,L,C]，默认C=1024、16头、单头d=64；L=N×(37×37+5)。Q/K/V整理为[B×16,L,64]。每头快速权重W0,W2:[B×16,64,d_h]、W1:[B×16,d_h,64]。

执行：Q/K先L2归一化，token学习率经softplus为正。Update计算Tθ(K)与V'的点积目标对W0/W1/W2的梯度，并做Muon式更新；Apply计算Tθ(Q)。离线训练时梯度穿过内层更新以学习QKV投影、学习率预测器和ShortConv，VGGT旧参数冻结。测试时慢权重不更新，每个场景只更新快速权重。

小例子：B=1、N=2，每图简化为4 patch+1前缀，则L=10；2头、d=3时Q/K/V是[2,10,3]。Update把10对(K,V')写入固定矩阵；Apply仍输出[2,10,3]，不会生成10×10注意力矩阵。

```
自行编写示意伪代码（非作者原代码）
q,k,v = project_and_l2norm(x)
v_target = shortconv2d(v)
fast = init_fast_weights()
for step in range(ttt_steps):
grad = grad_dot(T(fast,k), v_target)
fast = muon_update(fast, grad)
out = T(fast,q)
```

- FastWeightAttention与QKV/学习率，72-200行
- forward()，317-385行
- 快速权重梯度与同步，156-260行

## 6. 核心算法二：ShortConv2D空间目标

K和V都是x的线性投影，MLP可能只学WvWk⊠[1]。V为[B,Hd,L,d]；拆开前后缀，把patch变成[(B×N),(Hd×d),37,37]，逐通道3×3卷积，再恢复原序列。卷积慢权重由最终任务loss训练；其输出又充当内层TTT目标。camera/register token不卷积。

在简化2×2 patch中，卷积后的V'带相邻token统计，单点K必须预测邻域而非仅做线性换基。消融中对V做3×3优于5×5以及对K或KV同时卷积。

```
自行编写示意伪代码（非作者原代码）
patch,prefix,suffix = split_tokens(v)
grid = patch.reshape(B*N, heads*d, Hp, Wp)
v_prime = depthwise_conv3x3(grid)
v_prime = restore(v_prime,prefix,suffix)
```

ShortConv.forward()，22-61行；Q/K/V、卷积与归一化，169-200行。

## 7. 核心算法三：分块、分布式与查询式地图

总loss是局部token loss之和，因此每块梯度可独立求和。单GPU逐块加载QKV并累积梯度；多GPU对固定大小快速权重梯度all-reduce。建图map()清空并开启state tracking，缓存每层快速权重和camera token；query()只Apply，不compute_grad/update，新图不会污染地图。

论文/代码差异：正文说长序列默认2步；代码若num_ttt_steps=None使用2+global_num_imgs//1000，但infer()默认参数仍为1。复现必须显式固定步数。

分块，202-315行；infer()，255-364行；map/query，369-397行。

# 8. 数学公式逐式解释

## Softmax全局attention

$$o_i = \sum_j \mathrm{softmax}_j(q_i \cdot k_j / \sqrt{d}) v_j$$

q/k/v∈R^d；QK^T为L×L，softmax沿key维。L随图像数N线性增长，所以计算O(N²)。softmax分母依赖全部key，不能分块独立计算后简单相加。

## TTT目标

$$\theta^* = \arg\min_\theta \sum_i L(T_\theta(k_i), v_i), \quad o_i = T_{\theta^*}(q_i)$$

θ为固定大小快速权重。先写入K→V'，再以Q读取。论文使用点积目标；实现对点积求梯度并沿增大相似度方向更新。简化标量Tθ(k)=θk，k=2、v'=3、θ=1，点积6、梯度6；学习率0.1的上升更新得θ=1.6，q=4输出由4变6.4。此为自行直观解释。

## SwiGLU快速网络

$$T_\theta(x) = [\mathrm{SiLU}(xW_0) \odot (xW_2)]W_1$$

W0/W2将d映射到d_h，门控相乘后W1投回d，能容纳非线性K→V'。源码在ttt.py:88-118，手写反向在225-241行。

## 梯度可加性

$$\partial L_{\text{total}} / \partial \theta = \sum_s \sum_{i \in s} \partial L(k_i, v_i) / \partial \theta$$

每卡无需看到其他卡token，只同步固定MLP大小的梯度。替代softmax的关键是目标变为可分解。

## Muon正交化

$$X \leftarrow aX + (bA + cA^2)X, \quad A = XX^T$$

实现先归一化梯度，再做Newton-Schulz迭代，a=3.4445、b=-4.7750、c=2.0315，使更新方向近似谱归一。源码56-85行。

# 9. 实验与证据

- 1,000图：正文约58秒、摘要54秒；增强VGGT超过11分钟，11.6倍。两个时间按原文分别保留。
- DTU/ETH3D/NRGBD-D点图CD相对TTT3R降低约2-2.5倍；7Scenes-D仍弱于VGGT（0.030 vs 0.024）。
- ScanNet相机ATE 0.070，VGGT 0.035，不能称全面超过VGGT。
- ShortConv2D：CD改善0.008，mAA提升1.98个百分点；softmax上界仍为76.33。
- Wayspots 20cm/20°定位成功率2.94%→30.64%，提高27.70个百分点。

# 10. 局限与适用范围

- 作者报告：相机位姿、宽基线精度仍落后于softmax模型。
- 源码事实：训练harness公开，但README明确说数据集实现和预处理尚未公开，完整训练不能开箱复现。
- 工程：主代码/模型为NVIDIA OneWay Noncommercial，商业用途需核验。
- 推断：固定容量MLP可能在超大复杂场景饱和；增加步数改善优化，但不等同于增加容量。

## 11. 前置知识与50分钟路线

前置：VGGT交替attention；Q/K/V复杂度；TTT快/慢权重；SwiGLU；相机c2w与深度反投影。

- 0-8分钟：图1，写清KV为什么是场景记忆。
- 8-20分钟：式(2)-(5)，比较softmax与TTT可分块性。
- 20-32分钟：读FastWeightAttention.forward和手写反向。
- 32-42分钟：看表1/3/6，区分速度收益和精度代价。
- 42-50分钟：读map/query，设计VGGT接口替换。

思考题：为什么TTT梯度能分块求和，而softmax输出不能独立分块后相加?

## 12. 代码任务与面试题

优先文件：ttt_attention.py（token重排、学习率、操作序列）；ttt.py（SwiGLU、手写梯度、Muon）；models/vggt.py（输出接口、步数、map/query）。

最小路线：按README安装torch 2.7.1/cu126与项目，加载nvidia/vgg-ttt，先3图infer，再8/16/32/64/128图比较时间、峰值显存、CD与输出字段。官方未给单次推理最低显存；训练使用8×A100-80GB不是推理要求。

Q1：TTT-MLP为什么是场景表示？每层K→V'映射由当前场景的自监督梯度写入固定参数；Q经同一函数读出聚合信息。

Q2：ShortConv2D解决什么退化？K/V同源线性投影可能形成平凡映射；让V目标含邻域而K保持局部，迫使快速权重编码空间上下文。

# 13. 迁移到 VGGT / Depth Pro / YOLO

改动：保持YOLO和Depth Pro度量校准，把VGGT推理器换为VGG-T$^3$兼容接口，以pts3d/conf按bbox汇聚；长视频先map()，后续关键帧用query()定位。

预期：帧数增加时降低全局attention时延/显存，并让快速权重成为VLA Agent几何记忆。假设：相机位姿下降不能抵消效率收益；Depth Pro尺度校准可改善车辆远距误差。

最小对照：A=VGGT，B=VGG-T$^3$一步，C=VGG-T$^3$自适应步数，帧数8/32/128、统一518输入；报告≤30m/≤60m距离误差、ATE、点图CD、峰值显存和秒/帧；D=C+Depth Pro尺度。只有误差不显著下降且速度提升，才判断替换成立。